

# EMERGENCY RESPONSE SYSTEM



**CSE 221: Data Structure**

**Submitted by:**

| Name                | ID          | Intake |
|---------------------|-------------|--------|
| Md. Shaharia Islam  | 20255103200 | 55     |
| Inteshar Al Islam   | 20255103168 | 55     |
| Mujtahid Islam Abir | 20255103177 | 55     |

**Supervised by:**

**Talimul Bari Shreshtho**

*Lecturer,  
Department of CSE*

**Dept. of Computer Science & Engineering  
Bangladesh University of Business and Technology(BUBT)**

*April 29, 2026*

# Table of Contents

- Abstract
- Chapter 1: Introduction
  - 1.1 Problem Statement
  - 1.2 Objectives
  - 1.3 Flow of the Project
- Chapter 2: Background
  - 2.1 Existing System Analysis
  - 2.2 Supporting Concepts
- Chapter 3: System Analysis and Design
  - 3.1 Core Data Structures
  - 3.2 Data Flow
  - 3.3 Data Structure Summary
- Chapter 4: Implementation
  - 4.1 Interface Design
  - 4.2 Functional Modules
  - 4.3 Backend Logic
- Chapter 5: User Manual
  - 5.1 System Requirements
  - 5.2 User Operations
- Chapter 6: Conclusion
  - 6.1 Conclusion
  - 6.2 Limitations
  - 6.3 Future Work
- References

# Abstract

This project presents a **terminal-based Smart Emergency Response Management System** developed using **C++**, focusing on the application of core data structures. The system simulates how emergency services manage incoming cases, prioritize them based on severity, and assign ambulances efficiently.

Unlike traditional systems that rely on databases, this implementation uses multiple data structures such as **Priority Queue, Stack, Vector, Map, and Graph**. Each structure is chosen for its efficiency in handling specific operations.

The project demonstrates how combining different data structures can result in an optimized and realistic emergency management system.

# **Chapter 1: Introduction**

## **1.1 Problem Statement**

Managing emergency situations efficiently is a critical challenge. Without proper prioritization and resource allocation, response time increases, which can lead to serious consequences. A system is needed to handle emergencies based on severity and assign available ambulances effectively.

## **1.2 Objectives**

- To develop a terminal-based emergency system using C++
- To implement multiple data structures in a real-world scenario
- To prioritize emergencies based on severity
- To manage ambulance allocation efficiently
- To implement undo functionality

## **1.3 Flow of the Project**

The system starts with a menu-driven interface. The user inputs emergency details, which are stored and processed using appropriate data structures. Based on severity and location, the system assigns ambulances and updates status dynamically.

# **Chapter 2: Background**

## **2.1 Existing System Analysis**

Most modern emergency systems rely on complex databases and real-time tracking systems. While efficient, they do not clearly demonstrate the internal working of data structures. This project focuses on implementing core data structures directly in a simplified environment.

## **2.2 Supporting Concepts**

This project is based on fundamental data structure concepts including:

- Priority-based processing
- Stack operations for undo
- Dynamic arrays
- Graph-based distance calculation

# Chapter 3: System Analysis and Design

## 3.1 Core Data Structures

### Priority Queue (Max Heap)

Used to store emergency requests and always process the highest severity first.

Time Complexity:  $O(\log n)$

### Stack

Used for undo operations (Last-In-First-Out behavior).

Time Complexity:  $O(1)$

### Vector

Stores ambulance data and completed emergencies dynamically.

Time Complexity:  $O(1)$  access

### Map

Used for mapping area names and quick lookup.

Time Complexity:  $O(\log n)$

### Graph (Adjacency Matrix)

Represents distances between locations to find the nearest ambulance.

## 3.2 Data Flow

- Emergency input  $\rightarrow$  Priority Queue
- Assignment  $\rightarrow$  Graph + Vector
- Undo  $\rightarrow$  Stack
- Completed  $\rightarrow$  Vector

### 3.3 Data Structure Summary

| Feature            | Data Structure | Reason                 |
|--------------------|----------------|------------------------|
| Emergency Priority | Priority Queue | Highest severity first |
| Undo               | Stack          | LIFO behavior          |
| Ambulance Storage  | Vector         | Dynamic and fast       |
| Area Mapping       | Map            | Fast lookup            |
| Distance           | Graph          | Nearest selection      |

# Chapter 4: Implementation

## 4.1 Interface Design

The system uses a console-based menu-driven interface similar to the sample report .

## 4.2 Functional Modules

- `addEmergency()` → Adds emergency case
- `assignAmbulance()` → Assigns nearest ambulance
- `showPending()` → Displays pending cases
- `showCompleted()` → Displays handled cases
- `undo()` → Reverts last operation
- `returnAmbulance()` → Marks ambulance free

## 4.3 Backend Logic

All data is stored in memory using data structures. No external database or file system is used. The system ensures efficient operations using optimized algorithms.



# Chapter 5: User Manual

## 5.1 System Requirements

### Hardware:

- Any standard computer

### Software:

- Operating System: Windows/Linux/macOS
- Compiler: G++
- Environment: Terminal / Command Prompt

## 5.2 User Operations

### Main Menu:

User selects options from menu

```
==== EMERGENCY SYSTEM ====
1. Add Emergency
2. Assign Ambulance
3. Show All Ambulances
4. Hospital Status
5. Pending Emergencies
6. Completed Emergencies
7. System Stats
8. Undo
9. Return Ambulance
10. Exit
Choice:
```

### Add Emergency:

User inputs severity and location

```
Choice: 1
Enter Area (Mirpur/Dhanmondi/Gulshan/Uttara): mirpur
Enter Severity (1-5): 5
Emergency Added! ID: 1
```

### **Assign Ambulance:**

System automatically assigns based on priority

```
Choice: 2
```

```
Dispatch Successful  
Ambulance ID: 1  
Hospital: DMC  
Distance: 0 km
```

### **Undo:**

Reverts last action

```
Choice: 8  
Undo successful
```

### **Return Ambulance:**

Marks ambulance available again

```
Choice: 9  
Enter Ambulance ID: 1  
Ambulance 1 returned to MIRPUR
```

# Chapter 6: Conclusion

## 6.1 Conclusion

This project successfully demonstrates how multiple data structures can be integrated to build an efficient emergency response system. Each structure contributes to optimized performance and realistic behavior.

## 6.2 Limitations

- No persistent storage
- Static distance system
- No real-time tracking
- Console-based interface

## 6.3 Future Work

- Implement Dijkstra algorithm
- Add file/database storage
- Develop graphical interface
- Expand real-world mapping

## References

- Bjarne Stroustrup, *The C++ Programming Language*
- Mark Allen Weiss, *Data Structures and Algorithms*
- C++ Standard Library Documentation